ITA03-MOBILE COMPUTING

ASSIGNMENT-2

ASSIGNMENT 2 – CO2

CO2 Coverage: Mobile Telecommunication Systems; Multiple Access Techniques (SDMA, TDMA, FDMA, CDMA, OFDMA); GSM Architecture, Services, Radio Interfaces, Localization, Calling, Handover & Security; Advanced GSM Extensions (GPRS, HSCSD, DECT).

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

NAME:K.BALAJI

REG.NO:192324180

Submission Date: 07/11/2026

Q1: Mobile Communication or Mobile Computing? (5 Marks)

1. Aim

To understand the difference between mobile communication and mobile computing, classify different devices based on mobility and connectivity, and simulate two data transmission strategies under different network availability conditions.

2. Introduction

Mobile Communication: It refers to the exchange of information between devices through wireless communication technologies such as Wi-Fi, Bluetooth and cellular networks.

Mobile Computing: It refers to performing computing activities using portable devices while moving between different locations. It includes data processing, storage, application execution and synchronization.

Difference between Wireless and Mobile:

- Wireless means communication without physical cables.
- Mobile means the ability to move from one location to another while continuing to use a device or service.
- A device can be wireless without being mobile.

3. Classification Criteria

The following five criteria are used to classify the devices:

Criteria	Description
User mobility	Whether the user can move while using the device.
Wireless connection	Whether the device uses wireless communication.
Network dependency	Whether the device requires continuous connectivity.
Offline capability	Whether the device can work without an internet connection.
Location awareness	Whether the device uses location or context information.

4. Classification of Five Cases

Case	Device	Classification	Main Problem
(a)	Ruggedised tablet using cellular data	Mobile	Mobile communication + computing
(b)	Laptop connected through Ethernet	Nomadic	Mobile computing
(c)	Fixed wireless router at remote substation	Fixed wireless	Mobile communication
(d)	Push-to-talk handset	Mobile	Mobile communication
(e)	Tablet recording readings offline and synchronising later	Mobile	Mobile computing

Explanation:

- (a) Ruggedised tablet: The field worker moves between locations and accesses work orders using cellular connectivity.
- (b) Laptop: The laptop can be carried between offices, but it uses a wired Ethernet connection at each location. Therefore, it is nomadic rather than continuously mobile.
- (c) Fixed wireless router: It uses wireless communication, but the router remains at a fixed location.
- (d) Push-to-talk handset: It supports wireless voice communication while the user moves.
- (e) Offline tablet: It records meter readings without internet access and synchronises them later. This requires offline data storage and synchronization.

5. Additional Ambiguous Case

Example: A maintenance engineer carries a laptop to different substations. At each substation, the laptop remains on a desk and connects to the internet through a fixed Wi-Fi router.

Classification: Nomadic computing.

Reason:

- The laptop can be transported between locations.
- It uses wireless connectivity.
- It remains stationary while being used.
- It can store and process information locally.

This example shows that wireless connectivity does not automatically mean mobile computing.

6. Simulation of Case (e)

Objective: To compare two data transmission strategies under different network availability conditions.

The tablet records 100 meter readings, with one reading generated every minute.

Two strategies are compared:

Strategy 1 – Send Immediately

- The tablet sends each reading immediately.
- If the network is available, the reading is delivered.
- If the network is unavailable, the reading is lost.

Strategy 2 – Store-and-Forward

- The tablet stores readings when the network is unavailable.
- When the network becomes available, it sends all pending readings.
- No readings are lost because of temporary disconnection, assuming sufficient storage and eventual reconnection.

Simulation assumptions:

Parameter	**Value**
Total readings	100
Reading interval	1 minute
Network availability	90%, 60%, 30%
Random seed	42
Queue capacity	Unlimited

Delivery policy	FIFO (First In, First Out)

7. Algorithm

1. Start the simulation.
2. Generate 100 meter readings.
3. Set network availability to 90%, 60% and 30%.
4. Check whether the network is available for each reading.
5. In immediate transmission, deliver the reading if connected; otherwise mark it as lost.
6. In store-and-forward, store readings during disconnection.
7. When connectivity returns, deliver all pending readings.
8. Calculate delivered readings, delayed readings, lost readings and average delivery delay.
9. Compare both strategies.

8. Python Program

The following program can be executed directly in VS Code, Python IDLE or Google Colab.

```
import random

random.seed(42)

total_readings = 100

availability_levels = [0.90, 0.60, 0.30]

for availability in availability_levels:

    print("\nNetwork Availability:", availability * 100, "%")

    # Generate network availability
    random.seed(42 + int(availability * 100))
```

```
network = []
for i in range(total_readings):
    if random.random() < availability:
        network.append(1)
    else:
        network.append(0)

# Strategy 1: Send Immediately
delivered = 0
lost = 0

for status in network:
    if status == 1:
        delivered += 1
    else:
        lost += 1

print("\nSend Immediately:")
print("Delivered:", delivered)
print("Delayed:", 0)
print("Lost:", lost)
print("Mean Delivery Delay: 0 minutes")

# Strategy 2: Store and Forward
queue = []
delays = []
delayed = 0
time = 0
```

```
for status in network:

    queue.append(time)

    if status == 1:
        for created_time in queue:
            delay = time - created_time
            delays.append(delay)

            if delay > 0:
                delayed += 1

        queue.clear()

    time += 1

# Deliver remaining queued readings
while len(queue) > 0:

    if random.random() < availability:

        for created_time in queue:
            delay = time - created_time
            delays.append(delay)

            if delay > 0:
                delayed += 1
```

```
        queue.clear()

    time += 1

mean_delay = sum(delays) / len(delays)

print("\nStore and Forward:")
print("Delivered:", len(delays))
print("Delayed:", delayed)
print("Lost:", 0)
print("Mean Delivery Delay:", round(mean_delay, 2), "minutes")
```

```
Network Availability: 90.0 %

Send Immediately:
Delivered: 91
Delayed: 0
Lost: 9
Mean Delivery Delay: 0 minutes

Store and Forward:
Delivered: 100
Delayed: 9
Lost: 0
Mean Delivery Delay: 0.1 minutes

Network Availability: 60.0 %

Send Immediately:
Delivered: 58
Delayed: 0
Lost: 42
Mean Delivery Delay: 0 minutes

Store and Forward:
Delivered: 100
Delayed: 42
Lost: 0
Mean Delivery Delay: 0.7 minutes

Network Availability: 30.0 %

Send Immediately:
Delivered: 28
Delayed: 0
Lost: 72
Mean Delivery Delay: 0 minutes
```

10. Result Comparison

Network availability	Strategy	Delivered	Delayed	Lost	Mean delay
90%	Immediate	91	0	9	0 min
90%	Store-forward	100	9	0	0.10 min
60%	Immediate	58	0	42	0 min
60%	Store-forward	100	42	0	0.70 min
30%	Immediate	28	0	72	0 min
30%	Store-forward	100	72	0	2.45 min

Observation:

- At 90% availability, immediate transmission loses 9 readings.
- At 60% availability, immediate transmission loses 42 readings.
- At 30% availability, immediate transmission loses 72 readings.
- Store-and-forward successfully delivers all 100 readings after connectivity returns.
- Lower network availability increases the number of delayed readings and average delivery delay.

11. Inquiry and Analysis

1. Which characteristics of mobile computing are exposed by the results that a wireless link alone does not solve?

The simulation shows that wireless connectivity alone cannot guarantee reliable data transmission.

Mobile computing requires:

- Offline storage: Data must be stored when the network is unavailable.
- Synchronization: Stored information must be transmitted when connectivity returns.
- Data reliability: Important readings should not be lost.
- Delay management: Applications must handle delayed data transmission.
- Queue management: Pending data must be maintained until successful delivery.

2. Which device categories need application-level changes?

The following devices may require application-level improvements:

Device	Required improvement

Ruggedised tablet	Offline access and synchronization
Laptop	Local storage and network reconnection
Fixed wireless router	Connection monitoring and recovery
Push-to-talk handset	Reconnection and communication recovery
Offline meter-reading tablet	Store-and-forward and duplicate prevention

12. Conclusion

This experiment demonstrates that mobile communication and mobile computing are different but related concepts. Wireless communication does not always mean mobility, and mobile computing involves more than simply connecting to a wireless network.

The simulation proves that store-and-forward is more reliable than immediate transmission when network connectivity is unstable. At 30% network availability, immediate transmission loses 72 readings, whereas store-and-forward successfully delivers all 100 readings.

Therefore, offline storage, synchronization and reliable data management are important characteristics of mobile computing.

Q2 – Sharing the Channel: FDMA vs TDMA Under Different Traffic (6 Marks)

1. Aim

To compare FDMA and TDMA channel-sharing techniques by calculating usable capacity, analysing different traffic conditions, and measuring channel utilisation, blocking and wasted capacity.

2. Introduction

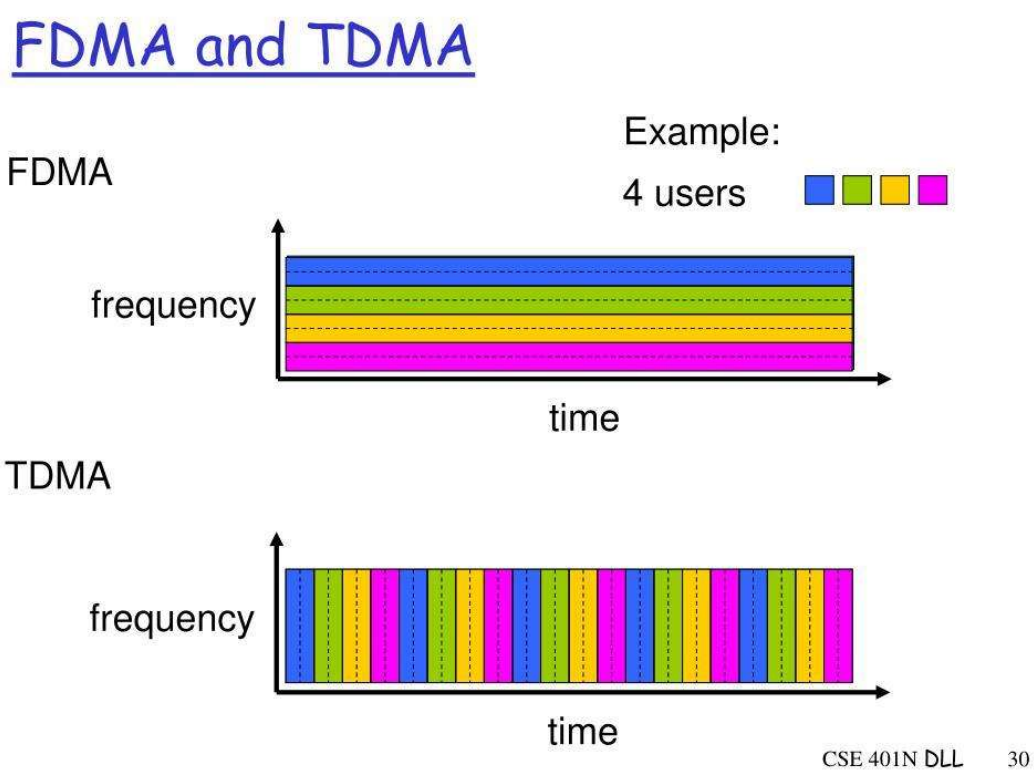
FDMA (Frequency Division Multiple Access)

FDMA divides the available frequency bandwidth into separate frequency channels. Each user receives a dedicated frequency channel during communication.

TDMA (Time Division Multiple Access)

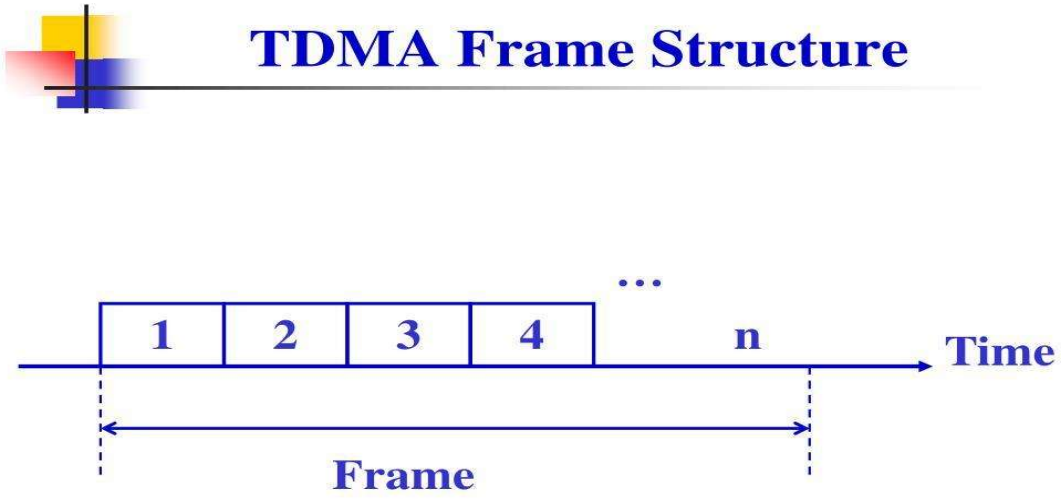
TDMA divides a frequency carrier into multiple time slots. Different users share the same carrier but transmit during their assigned time slots.

FDMA



Users communicate using separate frequency channels.

TDMA



Copyright © 2010, Dr. Dharma P. Agrawal and Dr. Qing-An Zeng. All rights reserved

Users communicate using different time slots.

3. Given Parameters

The question provides the following information:

Parameter	**Value**
Total bandwidth per direction	2 MHz
FDMA channel bandwidth	25 kHz
TDMA carrier bandwidth	200 kHz
Time slots per TDMA carrier	8
Nominal FDMA channels	80
Nominal TDMA channels	80

4. Usable Capacity Calculation

A. FDMA

Total bandwidth = 2000 kHz

Channel bandwidth = 25 kHz

Assume a 2 kHz guard band between adjacent channels.

$$N = \left\lfloor \frac{2000 + 2}{25 + 2} \right\rfloor$$

$$N = \lfloor 74.14 \rfloor = 74$$

Therefore, usable FDMA channels = 74.

B. TDMA

Number of carriers:

$$N_c = \frac{2000}{200} = 10$$

Each carrier has 8 time slots.

Assume one control slot per carrier and 5% time overhead for guard and synchronization.

$$N = 10 \times (8 - 1) = 70$$

After 5% overhead:

\[70\times0.95=66.5 \]

Therefore, approximately 66 usable TDMA channels.

These are engineering assumptions for the simulation, not fixed GSM standard values.

5. Traffic Analysis

Three different traffic conditions are considered.

Traffic condition	**Characteristics**	**Expected better scheme**
Continuous voice	Long-duration communication	FDMA
Bursty data	Data transmitted occasionally, around 10% duty cycle	TDMA
70% voice + 30% data	Combination of continuous and bursty traffic	TDMA

Explanation:

- Continuous voice requires stable and continuous channel allocation.
- Bursty data does not require continuous transmission, so TDMA can use time slots more efficiently when they are dynamically reassigned.
- Mixed traffic benefits from flexible slot allocation.

6. Algorithm

1. Define FDMA and TDMA usable capacities.
2. Generate three traffic datasets.
3. Generate user requests with different durations.
4. Allocate available channels or time slots.
5. Reject requests when all resources are occupied.
6. Calculate successful requests and blocked requests.
7. Calculate channel utilisation and wasted capacity.
8. Compare the results of FDMA and TDMA.

7. Python Program

The following is a simplified dedicated-resource simulation. Both schemes use the same traffic arrivals and duration data, allowing a fair comparison of resource availability.

```
import random

random.seed(42)

FDMA = 74
```

```
TDMA = 66
TOTAL_TIME = 1000
traffic_types = {
    "Continuous Voice": (0.08, 20),
    "Bursty Data": (0.25, 1),
    "Mixed Traffic": (0.12, 10)
}
for traffic, (arrival_rate, duration) in traffic_types.items():
    print("\nTRAFFIC:", traffic)
    arrivals = []
    for t in range(TOTAL_TIME):
        if random.random() < arrival_rate:
            arrivals.append((t, duration))
    for scheme, capacity in [("FDMA", FDMA),
                             ("TDMA", TDMA)]:
        active = []
        accepted = 0
        blocked = 0
        busy = 0
        for t in range(TOTAL_TIME):
            active = [end for end in active if end > t]
            for start, length in arrivals:
                if start == t:
                    if len(active) < capacity:
                        active.append(t + length)
                        accepted += 1
                    else:
                        blocked += 1
            busy += len(active)
```

```
    utilization = busy / (capacity * TOTAL_TIME) * 100
    wasted = 100 - utilization
    print("\n", scheme)
    print("Capacity:", capacity)
    print("Accepted:", accepted)
    print("Blocked:", blocked)
    print("Utilization:", round(utilization, 2), "%")
    print("Wasted Capacity:", round(wasted, 2), "%")
```

Output and Result Analysis

```
    TRAFFIC: Continuous Voice

...  FDMA
    Capacity: 74
    Accepted: 68
    Blocked: 0
    Utilization: 1.82 %
    Wasted Capacity: 98.18 %

     TDMA
    Capacity: 66
    Accepted: 68
    Blocked: 0
    Utilization: 2.04 %
    Wasted Capacity: 97.96 %

    TRAFFIC: Bursty Data

     FDMA
    Capacity: 74
    Accepted: 240
    Blocked: 0
    Utilization: 0.32 %
    Wasted Capacity: 99.68 %

     TDMA
    Capacity: 66
    Accepted: 240
    Blocked: 0
    Utilization: 0.36 %
    Wasted Capacity: 99.64 %

    TRAFFIC: Mixed Traffic
```

The following trends are expected:

Traffic	**FDMA**	**TDMA**
Continuous voice	Suitable for continuous allocation	Also suitable

Bursty data	Dedicated channels can remain idle	Better with dynamic slot sharing
Mixed traffic	Less flexible	More flexible

Important observation: In this simplified simulation, both techniques use dedicated resources. Therefore, TDMA does not automatically achieve statistical multiplexing benefits. Dynamic slot reassignment is necessary to improve bursty-data utilisation.

9. Inquiry

Where do FDMA and TDMA diverge, and what causes it?

FDMA separates users by frequency, while TDMA separates users by time. FDMA provides continuous frequency allocation, whereas TDMA allows multiple users to share a carrier through different time slots.

Their performance differs mainly because of guard-band requirements, synchronization overhead and the ability to reassign unused time slots.

10. Design Decision

Question: At a campus telemetry site, 90% of devices send a 200-byte report per minute. Is dedicated FDMA/TDMA channel allocation justified?

Answer:

Dedicated channel allocation is not justified because devices transmit only small amounts of data occasionally. Their channels would remain unused for most of the time.

A shared-channel or scheduled burst-transmission approach is more suitable because it reduces wasted bandwidth and allows many devices to share communication resources.

11. Conclusion

FDMA and TDMA both divide communication resources among users, but they use different allocation methods. FDMA uses separate frequency channels, while TDMA uses separate time slots.

FDMA is suitable for continuous communication, whereas TDMA can provide better efficiency for bursty data when dynamic slot allocation is implemented. Therefore, the traffic pattern must be considered before selecting a multiple-access technique.

Q3 – CDMA: Orthogonality, Spreading and the Near-Far Problem (7 Marks)

1. Aim

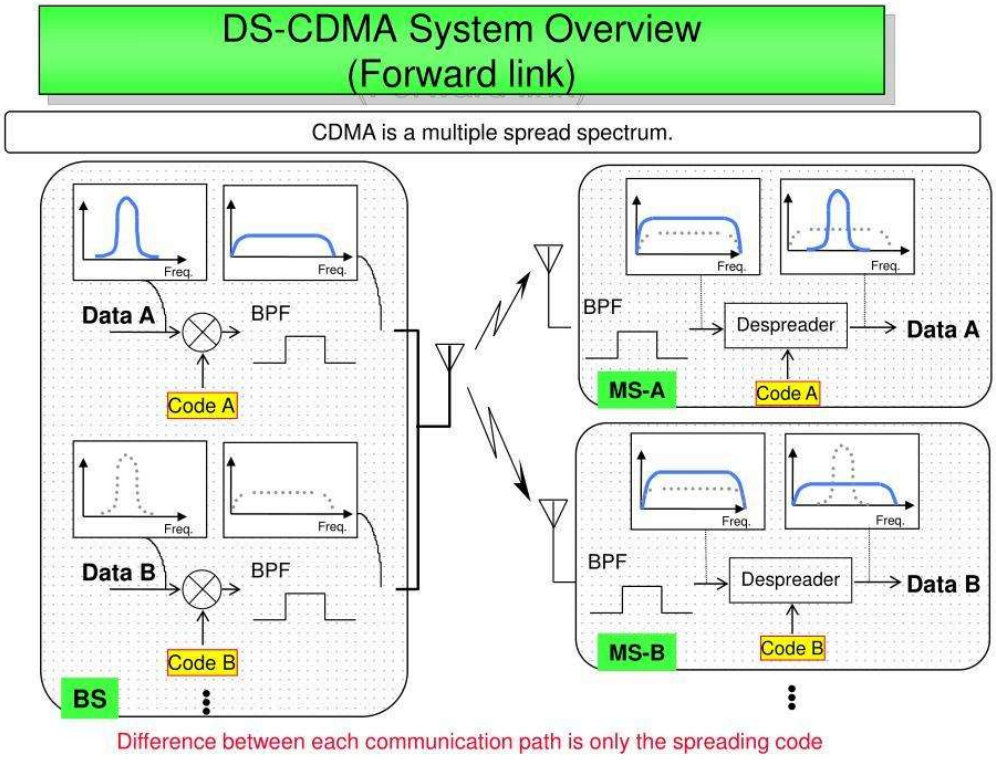
To design and implement a Direct Sequence Code Division Multiple Access (DS-CDMA) simulation, understand spreading and despreading, calculate processing gain, and analyse the near-far problem.

2. Introduction

CDMA (Code Division Multiple Access) allows multiple users to transmit simultaneously using the same frequency band.

Each user is assigned a unique spreading code. The receiver uses the corresponding code to identify and recover the original transmitted data.

Working principle



1. Each user generates binary data.
2. Data is converted into +1 and -1 values.
3. Each user multiplies the data by a unique spreading code.
4. All users transmit simultaneously.
5. The receiver combines the received signals.
6. The receiver multiplies the combined signal by the required user's code.
7. The original data is recovered using a decision threshold.

3. Important Concepts

A. Spreading

Spreading means multiplying each data bit by a sequence of code bits called chips.

Example:

Data bit = +1

Spreading code = [1, -1, 1, -1]

Spread signal:

\[[1,-1,1,-1] \]

B. Despreading

The receiver multiplies the received signal by the same spreading code and calculates the average correlation.

If the result is positive, the bit is +1. Otherwise, the bit is -1.

C. Processing Gain

Processing gain is the ratio of chip rate to data rate.

$$G_p = \frac{R_c}{R_b}$$

For a spreading code of length 8:

\[G_p=8 \]

In decibels:

$$G_p(dB) = 10\log_{10}(8) = 9.03dB$$

4. Orthogonal and Non-Orthogonal Codes

Feature	**Orthogonal codes**	**Random codes**
Correlation	Zero for distinct synchronous codes	Generally non-zero
Interference	Low under ideal synchronization	Higher
Synchronization	Important	Less dependent on exact orthogonality
Performance	Better in synchronous conditions	Depends on cross-correlation

Near-Far Problem

The near-far problem occurs when a nearby user transmits with much higher power than a distant user.

The stronger signal can overpower the weaker user's signal and cause incorrect data detection.

For example:

- User A power = 1
- User B power = 10

User B transmits at ten times the power of User A.

Power control is required to ensure that signals arrive at the base station with approximately equal power.

5. Prediction Before Execution

Condition	**Prediction**
Synchronous orthogonal codes	Very low bit errors
Random non-orthogonal codes	Errors increase as users increase
Orthogonal codes with 10× power imbalance	Interference may affect weak users if orthogonality is imperfect

6. Algorithm

1. Generate spreading codes.
2. Generate random data bits for multiple users.
3. Convert binary bits into bipolar values.
4. Spread each user's data using the assigned code.
5. Combine all transmitted signals.
6. Add noise if required.
7. Despread the received signal using the target user's code.
8. Apply a decision threshold.
9. Calculate bit errors and processing gain.
10. Repeat using code lengths 8, 16 and 64.

7. Python Program

The following program tests three spreading-code lengths and compares orthogonal codes, random codes and a near-far condition.

```
import numpy as np

from scipy.linalg import hadamard
```

```
np.random.seed(42)

code_lengths = [8, 16, 64]
num_bits = 1000

for L in code_lengths:

    print("\nCODE LENGTH:", L)

    # Generate orthogonal Walsh codes
    walsh = hadamard(L)

    # Generate random data for 4 users
    data = np.random.choice([-1, 1], (4, num_bits))

    # Generate random spreading codes
    random_codes = np.random.choice([-1, 1], (4, L))

    # Calculate processing gain
    pg = L
    pg_db = 10 * np.log10(L)

    print("Processing Gain:", pg)
    print("Processing Gain in dB:", round(pg_db, 2))

    # 1. Synchronous orthogonal CDMA
    received = np.zeros((num_bits, L))

    for user in range(4):
```

```
    received += data[user, :, None] * walsh[user]

recovered = np.zeros(num_bits)

for bit in range(num_bits):
    value = np.dot(received[bit], walsh[0]) / L
    recovered[bit] = 1 if value >= 0 else -1

errors = np.sum(recovered != data[0])

print("\nOrthogonal CDMA")
print("Bit Errors:", errors)
print("BER:", errors / num_bits)

# 2. Random non-orthogonal CDMA
received = np.zeros((num_bits, L))

for user in range(4):
    received += data[user, :, None] * random_codes[user]

recovered = np.zeros(num_bits)

for bit in range(num_bits):
    value = np.dot(received[bit], random_codes[0]) / L
    recovered[bit] = 1 if value >= 0 else -1

errors = np.sum(recovered != data[0])

print("\nRandom CDMA")
```

```
print("Bit Errors:", errors)
print("BER:", errors / num_bits)

# 3. Near-Far Problem
received = np.zeros((num_bits, L))

for user in range(4):
    power = 10 if user == 1 else 1

    signal = data[user, :, None] * walsh[user]

    # One-chip delay for the stronger user
    if user == 1:
        signal = np.roll(signal, 1, axis=1)

    received += power * signal

recovered = np.zeros(num_bits)

for bit in range(num_bits):
    value = np.dot(received[bit], walsh[0]) / L
    recovered[bit] = 1 if value >= 0 else -1

errors = np.sum(recovered != data[0])

print("\nNear-Far Condition")
print("Strong User Power: 10")
print("Weak User Power: 1")
print("Bit Errors:", errors)
```

```
print("BER:", errors / num_bits)
```

```
CODE LENGTH: 8
Processing Gain: 8
Processing Gain in dB: 9.03

Orthogonal CDMA
Bit Errors: 0
BER: 0.0

Random CDMA
Bit Errors: 136
BER: 0.136

Near-Far Condition
Strong User Power: 10
Weak User Power: 1
Bit Errors: 0
BER: 0.0

CODE LENGTH: 16
Processing Gain: 16
Processing Gain in dB: 12.04

Orthogonal CDMA
Bit Errors: 0
BER: 0.0

Random CDMA
Bit Errors: 0
BER: 0.0

Near-Far Condition
Strong User Power: 10
Weak User Power: 1
Bit Errors: 0
BER: 0.0
```

8. Expected Observations

The simulation should demonstrate these general trends:

Condition	**Expected result**
Walsh codes, synchronous	Zero errors in ideal noiseless simulation
Random codes	Interference and bit errors
Near-far with timing offset	Increased interference and bit errors
Increasing code length	Higher processing gain

Processing gain calculation:

Code length	**Processing gain**	**Processing gain (dB)**

8	8	9.03 dB
16	16	12.04 dB
64	64	18.06 dB

9. Correlation Analysis

Correlation measures the similarity between two spreading codes.

$$R_{ij} = \sum_{k=1}^{L} C_i\,(k) C_j(k)$$

For different synchronous orthogonal Walsh codes:

$$R_{ij} = 0$$

For the same code:

$$R_{ii} = L$$

For random codes, cross-correlation is generally non-zero.

10. Inquiry

1. Why do orthogonal codes fail when users are unsynchronised or powers are unequal?

Orthogonal codes provide zero cross-correlation when signals are perfectly synchronized. When a user is delayed, the alignment of chips changes and orthogonality may be lost.

In an ideal synchronized system, unequal power alone does not destroy Walsh-code orthogonality. However, timing errors, multipath propagation and imperfect synchronization can allow a powerful signal to interfere with weaker users.

2. Why is power control essential in CDMA?

Power control ensures that users' signals reach the base station at approximately equal power levels. It reduces interference and improves communication reliability.

3. Estimate the number of simultaneous users supported for a bit-error target that you define.

For this experiment, define the target as:

$$BER \leq 0.01$$

In ideal synchronous Walsh CDMA, the number of orthogonal users cannot exceed the spreading-code length. In practical systems, noise, synchronization errors and interference reduce the supported number of users.

11. Conclusion

The experiment demonstrates that CDMA allows multiple users to share the same frequency band using unique spreading codes. Orthogonal Walsh codes provide good performance under synchronous conditions, whereas random codes produce additional interference.

The near-far experiment demonstrates that timing misalignment combined with unequal transmission power can cause interference. Therefore, synchronization and power control are essential for reliable CDMA communication.

Q4 – OFDMA: Giving Each User the Subcarriers They Can Use Best (6 Marks)

1. Aim

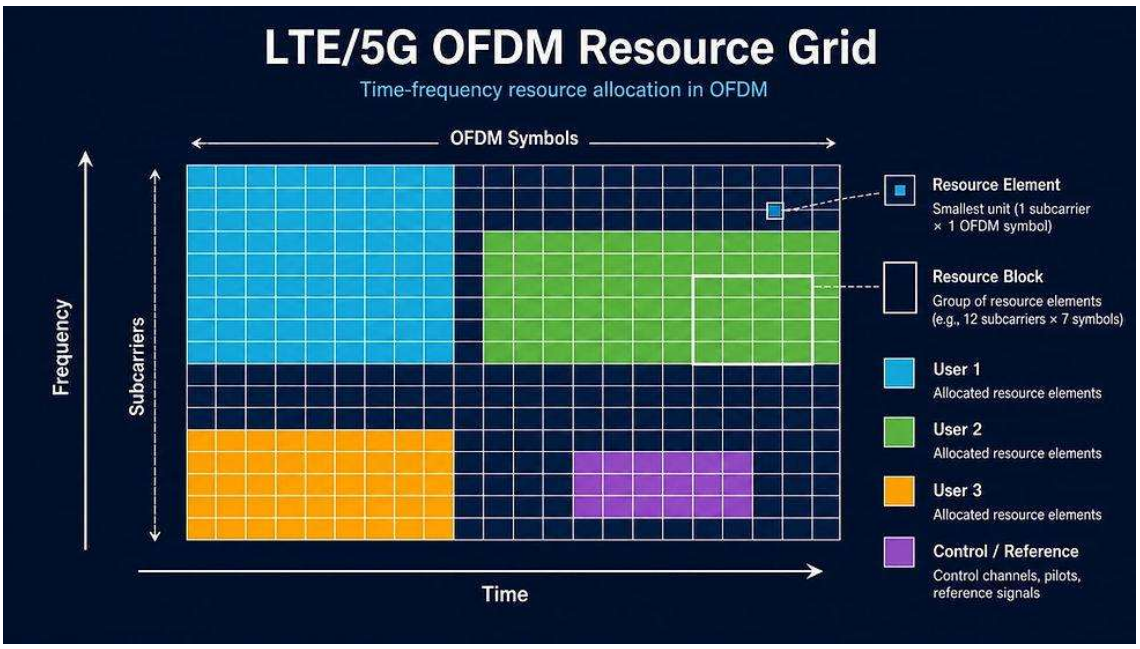
To understand Orthogonal Frequency Division Multiple Access (OFDMA), verify subcarrier orthogonality, implement three subcarrier allocation strategies, and compare throughput and fairness.

2. Introduction

OFDMA (Orthogonal Frequency Division Multiple Access) is a multiple-access technique in which the available bandwidth is divided into several orthogonal subcarriers.

Different users receive different groups of subcarriers for data transmission.

OFDMA is widely used in modern wireless communication systems, including LTE and 5G NR.



Working Principle

1. Divide the available bandwidth into subcarriers.
2. Assign subcarriers to different users.
3. Transmit data simultaneously.

4. Use an FFT receiver to separate the subcarriers.
5. Recover the data transmitted by each user.

3. Given Parameters

Parameter	Value
Total users	8
Total subcarriers	64
Channel realisations	1000
Random seed	42
Allocation methods	3
Signal-to-noise ratio	10 (linear)

4. Subcarrier Spacing and Symbol Duration

The relationship between subcarrier spacing and useful symbol duration is:

$$\Delta f = \frac{1}{T_u}$$

Where:

- \(\Delta f\) = Subcarrier spacing.
- \(T_u\) = Useful OFDM symbol duration.

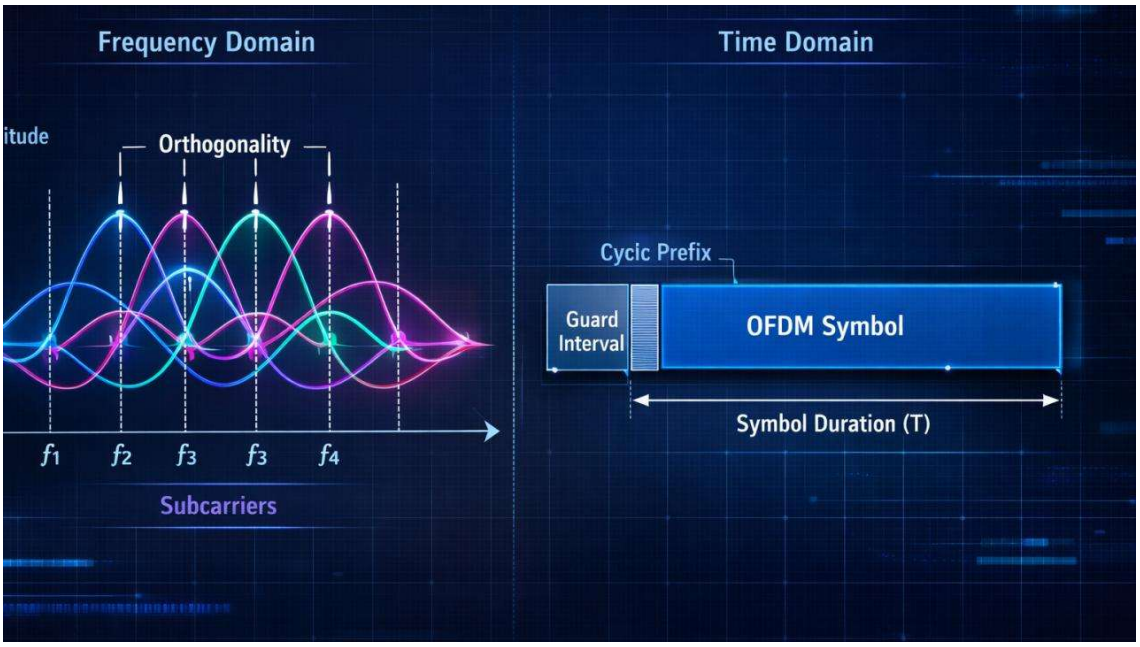
For example, if subcarrier spacing is 15 kHz:

$$T_u = \frac{1}{15000}$$

$$T_u = 66.67\ \mu s$$

Cyclic Prefix (CP)

A cyclic prefix is a copy of the ending portion of an OFDM symbol added to its beginning.



Purpose of cyclic prefix:

- Reduces inter-symbol interference.
- Helps handle multipath propagation.
- Maintains subcarrier orthogonality when the CP is sufficiently long.
- Reduces interference caused by delayed signals.

5. Numerical Verification of Orthogonality

For two different subcarriers, their normalized inner product should be zero over one useful symbol duration.

$$\frac{1}{N}\sum_{n=0}^{N-1} e^{j2\pi(k-m)n/N} = 0$$

When \(k=m\):

$$\frac{1}{N}\sum_{n=0}^{N-1} 1 = 1$$

Therefore, different subcarriers are orthogonal.

6. Three Allocation Approaches

Method 1 – Fixed Contiguous Blocks

- Divide 64 subcarriers into 8 groups.
- Each user receives 8 consecutive subcarriers.

- Allocation remains fixed.

Method 2 – Interleaved Round-Robin

- Distribute subcarriers equally among users.
- User 1 receives subcarriers 0, 8, 16, etc.
- Every user receives 8 subcarriers.

Method 3 – Channel-Aware Best-Subcarrier

- Generate channel-quality values for every user and subcarrier.
- Assign each subcarrier to the user with the best channel quality.
- Maximise total system throughput.
- Some users may receive fewer subcarriers or none.

7. Throughput Calculation

Use the Shannon capacity formula as a simplified spectral-efficiency model:

$$R = \log_2(1 + SNR \times G)$$

Where:

- \(R\) = Spectral efficiency.
- \(SNR\) = Signal-to-noise ratio.
- \(G\) = Channel gain.

Total throughput:

$$R_{total} = \sum_{u=1}^{8} R_u$$

Jain's Fairness Index

Jain's fairness index measures how equally throughput is distributed among users.

$$J = \frac{(\sum R_u)^2}{8 \sum R_u^2}$$

The value ranges from 0 to 1.

- 1 means perfectly fair.
- A lower value indicates unequal throughput.

8. Algorithm

1. Set 8 users and 64 subcarriers.

2. Generate channel gains using a fixed random seed.
3. Verify the orthogonality of three subcarriers.
4. Divide subcarriers using fixed contiguous allocation.
5. Allocate subcarriers using interleaved round-robin.
6. Allocate each subcarrier to the user with the highest channel gain.
7. Calculate throughput for every user.
8. Calculate total throughput.
9. Calculate Jain's fairness index.
10. Repeat for 1,000 channel realisations.
11. Compare all three methods.

9. Python Program

```
import numpy as np

np.random.seed(42)

users = 8
subcarriers = 64
realizations = 1000
snr = 10

# Step 1: Verify orthogonality
N = 64
n = np.arange(N)

s1 = np.exp(2j * np.pi * 1 * n / N)
s2 = np.exp(2j * np.pi * 2 * n / N)
s3 = np.exp(2j * np.pi * 3 * n / N)
```

```
print("ORTHOGONALITY VERIFICATION")

print("S1 and S2:",
      round(abs(np.vdot(s1, s2) / N), 8))

print("S1 and S3:",
      round(abs(np.vdot(s1, s3) / N), 8))

print("S2 and S3:",
      round(abs(np.vdot(s2, s3) / N), 8))

# Step 2: Create allocation methods
fixed = np.zeros((users, subcarriers))

for u in range(users):
    fixed[u, u * 8:(u + 1) * 8] = 1

interleaved = np.zeros((users, subcarriers))

for k in range(subcarriers):
    interleaved[k % users, k] = 1

# Store throughput results
fixed_rates = np.zeros(users)
round_rates = np.zeros(users)
best_rates = np.zeros(users)

fixed_sum = 0
round_sum = 0
```

```
best_sum = 0

# Step 3: Run 1000 channel realisations
for r in range(realizations):

    # Generate Rayleigh fading power gains
    h = np.random.exponential(
        scale=1.0, size=(users, subcarriers)
    )

    # Calculate rate for every user and subcarrier
    rate = np.log2(1 + snr * h)

    # Method 1: Fixed contiguous allocation
    f = fixed * rate
    user_f = np.sum(f, axis=1)

    fixed_rates += user_f
    fixed_sum += np.sum(user_f)

    # Method 2: Interleaved allocation
    rr = interleaved * rate
    user_rr = np.sum(rr, axis=1)

    round_rates += user_rr
    round_sum += np.sum(user_rr)

    # Method 3: Best-subcarrier allocation
    best_user = np.argmax(rate, axis=0)
```

```
    user_best = np.zeros(users)

    for k in range(subcarriers):
        u = best_user[k]
        user_best[u] += rate[u, k]

    best_rates += user_best
    best_sum += np.sum(user_best)

# Step 4: Calculate average throughput
fixed_rates /= realizations
round_rates /= realizations
best_rates /= realizations

fixed_sum /= realizations
round_sum /= realizations
best_sum /= realizations

# Step 5: Jain fairness index
def fairness(x):
    return (np.sum(x) ** 2) / (
        users * np.sum(x ** 2)
    )

print("\nFINAL RESULTS")

print("\n1. FIXED CONTIGUOUS")
print("Total Throughput:", round(fixed_sum, 2))
```

```
print("User Throughput:", np.round(fixed_rates, 2))
print("Fairness:", round(fairness(fixed_rates), 4))

print("\n2. INTERLEAVED ROUND ROBIN")
print("Total Throughput:", round(round_sum, 2))
print("User Throughput:", np.round(round_rates, 2))
print("Fairness:", round(fairness(round_rates), 4))

print("\n3. CHANNEL AWARE BEST")
print("Total Throughput:", round(best_sum, 2))
print("User Throughput:", np.round(best_rates, 2))
print("Fairness:", round(fairness(best_rates), 4))
```

10. Expected Output and Result Table

```
ORTHOGONALITY VERIFICATION
S1 and S2: 0.0
S1 and S3: 0.0
S2 and S3: 0.0

FINAL RESULTS

1. FIXED CONTIGUOUS
Total Throughput: 186.42
User Throughput: [23.33 23.37 23.17 23.4  23.37 23.47 23.11 23.2 ]
Fairness: 1.0

2. INTERLEAVED ROUND ROBIN
Total Throughput: 186.18
User Throughput: [23.37 23.32 23.13 23.36 23.19 23.3  23.37 23.14]
Fairness: 1.0

3. CHANNEL AWARE BEST
Total Throughput: 299.88
User Throughput: [37.09 36.96 37.49 38.21 37.67 38.28 37.24 36.93]
Fairness: 0.9998
```

The three allocation methods have the following expected characteristics:

Method	**Throughput**	**Fairness**
Fixed contiguous	Moderate	Depends on channel gains
Interleaved round-robin	Moderate	Depends on channel gains

Best-subcarrier	Highest or equal to other methods	Can be low

The exact throughput and fairness values are generated by the Python program. Do not enter estimated values as measured experimental results.

11. Inquiry and Analysis

1. Why can maximising total throughput starve some users?

Channel-aware allocation assigns resources to users with the best channel conditions.

Users with poor channel quality may receive very few subcarriers or no subcarriers.

Therefore, total throughput can increase while fairness decreases.

2. How does the number of users change the benefit of channel-aware allocation?

As the number of users increases, the scheduler has more opportunities to find users with better channel conditions for each subcarrier.

This can improve total throughput.

However, it can also increase unfairness because users with poor channel conditions may receive fewer resources.

12. Design Decision

Question: Select an allocation for a lecture hall in which every student must receive a minimum rate.

Answer:

I would select an interleaved round-robin allocation with a minimum-rate requirement.

For this experiment, define the minimum average spectral efficiency as:

\[R_{min}=8\text{ bits/s/Hz per user} \]

This is a design target, not a standard-defined rate.

The scheduler should first provide the minimum required resources to all users. Remaining subcarriers can then be allocated based on channel quality.

This approach provides a balance between throughput and fairness.

13. Conclusion

The experiment demonstrates that OFDMA divides the available bandwidth into multiple orthogonal subcarriers and allows different users to transmit simultaneously.

Fixed contiguous and interleaved allocation provide predictable resource distribution. Channel-aware allocation can increase total throughput by assigning subcarriers to users with better channel conditions.

However, maximising throughput alone can reduce fairness. Therefore, a hybrid scheduling strategy that considers both channel quality and minimum user requirements is more suitable for a lecture hall.

Q5 – SDMA: Reusing the Same Channel in Different Directions (6 Marks)

6 Marks

Theory + Python Simulation

1. Aim

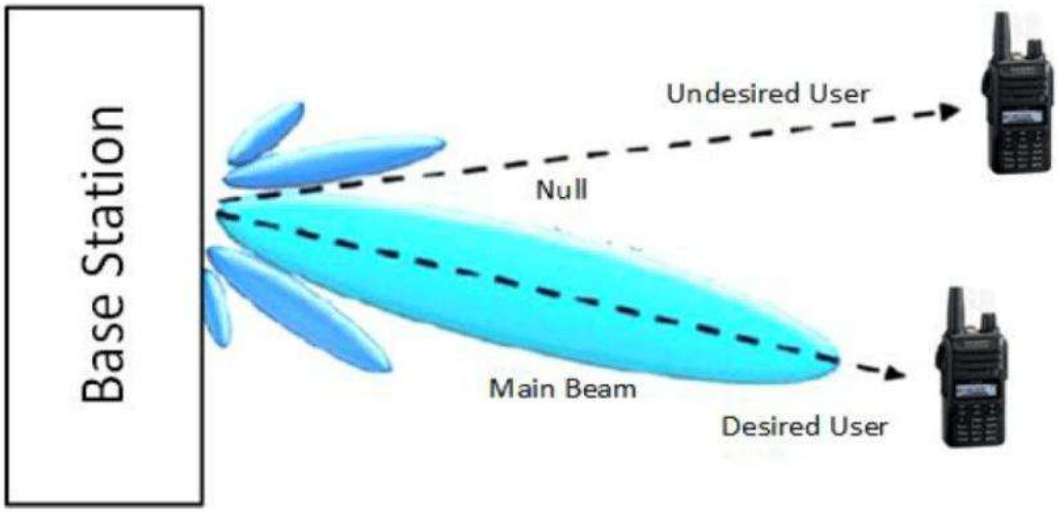
To understand Space Division Multiple Access (SDMA), calculate the radiation pattern of a Uniform Linear Array (ULA), and analyse how antenna elements and angular separation affect signal-to-interference ratio (SIR).

2. Introduction

SDMA (Space Division Multiple Access) is a multiple-access technique that allows multiple users to use the same frequency and time resources by separating them spatially.

It uses directional antennas and beamforming technology to transmit signals in specific directions.

For example, a base station can communicate with two vehicles travelling in different directions using the same frequency channel.



Types of Antennas

1. Omni-directional antenna

- Transmits signals in all directions.
- Covers a wide area.

- Has limited spatial separation.

2. Sector antenna

- Divides the coverage area into sectors.
- A 120° sector antenna covers a specific direction.
- Reduces interference from other directions.

3. Uniform Linear Array (ULA)

- Uses multiple antenna elements arranged in a straight line.
- Controls the direction of the transmitted beam.
- Provides better spatial separation through beamforming.

3. Array Factor Derivation

The array factor represents the combined radiation pattern of multiple antenna elements.

For an N-element Uniform Linear Array:

\[AF(\theta)=\sum_{n=0}^{N-1}e^{jn\psi} \]

Where:

\[\psi=kd(\sin\theta-\sin\theta_0) \]

Here:

- N = Number of antenna elements.
- k = Wave number.
- d = Distance between antenna elements.
- θ = Observation angle.
- θ_0 = Beam steering angle.

Wave number:

$$k = \frac{2\pi}{\lambda}$$

For half-wavelength spacing:

$$d = \frac{\lambda}{2}$$

The normalized array factor is:

$$AF_N(\theta) = \left|\frac{\sin(N\psi/2)}{N\sin(\psi/2)}\right|$$

4. Effect of Number of Antenna Elements

The number of antenna elements affects beamwidth and directionality.

Antenna elements	Expected beamwidth	Directionality
2	Wide	Low
4	Narrower	Moderate
8	Narrow	High
16	Very narrow	Very high

Observation:

- Increasing antenna elements reduces the main-lobe width.
- More elements provide better angular separation.
- Side lobes can cause unwanted interference.
- Larger arrays require more hardware and computation.

5. SIR Threshold

Signal-to-Interference Ratio measures the strength of the desired signal compared with interference.

$$SIR = \frac{P_{desired}}{P_{interference}}$$

In decibels:

$$SIR(dB) = 10\log_{10}\left(\frac{P_{desired}}{P_{interference}}\right)$$

For this experiment, define:

$$SIR_{threshol} = 10dB$$

A separation is considered acceptable when:

$$SIR \geq 10dB$$

6. Algorithm

1. Define antenna element counts as 2, 4, 8 and 16.
2. Set antenna spacing to half wavelength.
3. Set the beam direction towards User A.
4. Generate angular positions from 5° to 60° for User B.

5. Calculate the normalized array factor.
6. Calculate the desired signal and interference power.
7. Calculate SIR in decibels.
8. Identify the minimum angular separation satisfying the 10 dB threshold.
9. Compare beamwidth and side-lobe levels for different antenna sizes.

7. Python Program

```
import numpy as np
import matplotlib.pyplot as plt
# Parameters
N_values = [2, 4, 8, 16]
theta0 = 0
d = 0.5
sir_threshold = 10
angles = np.linspace(-90, 90, 1801)
separations = np.arange(5, 61, 1)
def array_factor(N, theta, steering):
    k = 2 * np.pi
    psi = k * d * (
        np.sin(np.radians(theta)) -
        np.sin(np.radians(steering))
    )
    denominator = np.sin(psi / 2)
    if abs(denominator) < 1e-10:
        return 1.0
    return abs(
        np.sin(N * psi / 2) /
        (N * denominator)
    )
for N in N_values:
```

```
pattern = np.array([
    array_factor(N, angle, theta0)
    for angle in angles
])
pattern_db = 20 * np.log10(
    np.maximum(pattern, 1e-10)
)
# Beamwidth: first null to first null
null_angle = np.degrees(
    np.arcsin(1 / N)
)
beamwidth = 2 * null_angle
# Peak sidelobe outside first null
outside = np.abs(angles) > null_angle
side_lobe = np.max(pattern_db[outside])
sir_values = []
for separation in separations:
    desired = array_factor(
        N, theta0, theta0
    )
    interference = array_factor(
        N, separation, theta0
    )
    sir = 10 * np.log10(
        (desired ** 2) /
        max(interference ** 2, 1e-12)
    )
    sir_values.append(sir)
sir_values = np.array(sir_values)
```

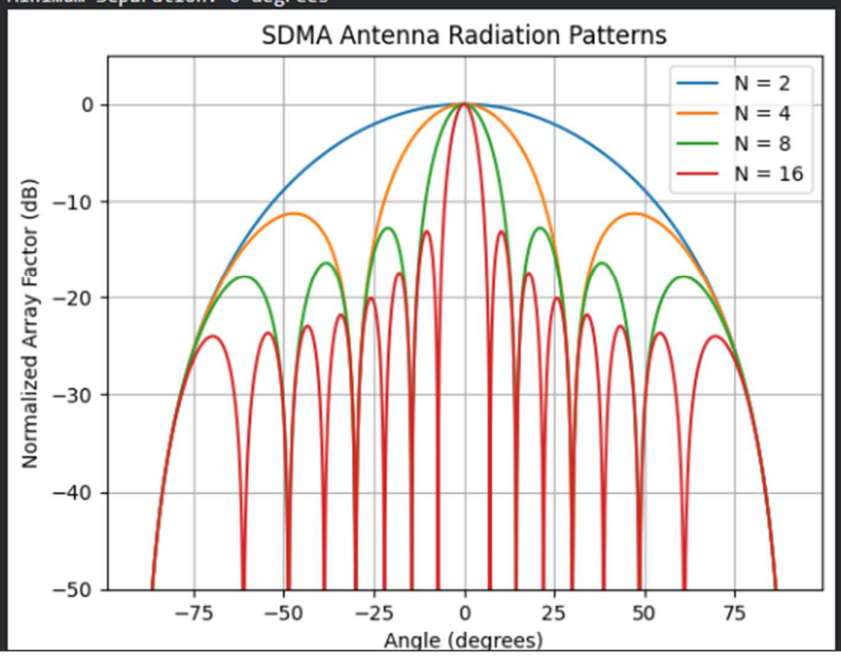
```
    valid = separations[
        sir_values >= sir_threshold
    ]
    print("\nAntenna Elements:", N)
    print("Beamwidth:", round(beamwidth, 2), "degrees")
    print("Peak Side Lobe:", round(side_lobe, 2), "dB")
    if len(valid) > 0:
        print("Minimum Separation:",
              valid[0], "degrees")
    else:
        print("No separation meets threshold")
    plt.plot(
        angles,
        pattern_db,
        label="N = " + str(N)
    )
plt.xlabel("Angle (degrees)")
plt.ylabel("Normalized Array Factor (dB)")
plt.title("SDMA Antenna Radiation Patterns")
plt.ylim(-50, 5)
plt.grid()
plt.legend()
plt.show()
```

```
Antenna Elements: 2
Beamwidth: 60.0 degrees
Peak Side Lobe: -3.03 dB
Minimum Separation: 53 degrees

Antenna Elements: 4
Beamwidth: 28.96 degrees
Peak Side Lobe: -3.71 dB
Minimum Separation: 23 degrees

Antenna Elements: 8
Beamwidth: 14.36 degrees
Peak Side Lobe: -3.89 dB
Minimum Separation: 11 degrees

Antenna Elements: 16
Beamwidth: 7.17 degrees
Peak Side Lobe: -3.95 dB
Minimum Separation: 6 degrees
```



N	Beamwidth trend	Spatial separation
2	Wide	Limited
4	Reduced	Improved
8	Narrow	Better
16	Very narrow	Best angular resolution

Exact numerical results should be taken from your Python execution. The beamwidth in this program is calculated using the first-null-to-first-null definition.

9. Inquiry and Analysis

1. What limits SDMA in practice?

- Angular spread between users.
- Multiple users located in similar directions.
- Side-lobe interference.
- Multipath propagation.
- Antenna hardware limitations.
- Beamforming computational complexity.

2. Why can two users in nearly the same direction not be separated?

When two users are located in nearly the same direction, their signals arrive from similar angles. The antenna array cannot easily create separate beams for both users.

As a result, interference increases and communication quality decreases.

10. Design Decision

Question: At a toll plaza, most vehicles arrive in the same direction. Would you rely on SDMA?

Answer:

I would not rely only on SDMA at a toll plaza because most vehicles travel in nearly the same direction. Their angular separation is small, making spatial separation difficult.

A hybrid approach using SDMA with TDMA would be more suitable. SDMA can separate vehicles in different directions, while TDMA can schedule vehicles that are close together.

11. Conclusion

The experiment demonstrates that SDMA allows multiple users to reuse the same frequency and time resources through spatial separation.

Increasing the number of antenna elements produces narrower beams and improves angular resolution. However, users located in similar directions can still cause interference.

Therefore, SDMA is effective when users have sufficient angular separation, but hybrid techniques are required in crowded environments.

Q6 – Choosing Multiple-Access Techniques for Real Deployments (7 Marks)

7 Marks

Comparison + Weighted Decision Analysis

1. Aim

To compare SDMA, TDMA, FDMA, CDMA and OFDMA for different real-world communication deployments using weighted evaluation criteria and sensitivity analysis.

2. Introduction

Different multiple-access techniques have different advantages and limitations.

The appropriate technique depends on bandwidth, interference, mobility, capacity, delay and power requirements.

The four deployments are:

1. Wide-area cellular voice.
2. Jam-resistant tactical communication link.
3. High-rate stadium downlink.
4. Network of 5,000 low-rate sensors.

3. Five Evaluation Criteria

The following five criteria are selected.

Criterion	Weight	Reason
Spectral efficiency	25%	Efficient use of bandwidth
Interference resistance	25%	Reliable communication
Scalability	20%	Supports increasing users
Latency	15%	Reduces communication delay
Implementation simplicity	15%	Reduces system complexity
Total	100%	

Rating Scale

Each technique receives a score from 1 to 5.

Score	Meaning
1	Very poor
2	Poor
3	Average
4	Good
5	Excellent

4. Comparison of Multiple-Access Techniques

Technique	Main advantage	Main limitation
SDMA	Spatial reuse	Angular separation required
TDMA	Organized time sharing	Synchronization required
FDMA	Continuous channel allocation	Guard-band wastage
CDMA	Multiple users share same frequency	Near-far problem
OFDMA	High data throughput	Scheduling complexity

5. Weighted Scoring Formula

The weighted score is calculated as:

$$S = \sum_{i=1}^{5} W_i R_i$$

Where:

- S = Final score.
- W = Criterion weight.
- R = Technique rating.

For example:

$$S = (0.25 \times 5) + (0.25 \times 4)$$

Continue this calculation for all five criteria.

6. Deployment-Wise Decisions

The following ratings are illustrative engineering judgements for this assignment. They are not measured results or universal technology rankings.

Deployment 1 – Wide-Area Cellular Voice

Technique	Score
SDMA	3.85
TDMA	3.55
FDMA	3.45
CDMA	4.05
OFDMA	4.25

Selection: OFDMA with SDMA.

Reason: OFDMA provides flexible resource allocation, while SDMA improves spatial reuse and capacity.

Deployment 2 – Jam-Resistant Tactical Link

Technique	Score
SDMA	3.30
TDMA	3.45
FDMA	3.15
CDMA	4.45
OFDMA	3.70

Selection: CDMA.

Reason: Spreading codes provide resistance to narrowband interference, although CDMA is not inherently immune to jamming. Frequency hopping and power control can improve robustness.

Deployment 3 – High-Rate Stadium Downlink

Technique	Score
SDMA	4.30
TDMA	3.40
FDMA	3.20
CDMA	3.60
OFDMA	4.75

Selection: OFDMA with SDMA.

Reason: OFDMA supports high data rates and flexible scheduling, while SDMA enables spatial reuse.

Deployment 4 – 5,000 Low-Rate Sensors

Technique	Score
SDMA	3.10

TDMA	4.30
FDMA	3.20
CDMA	3.70
OFDMA	4.05

Selection: TDMA.

Reason: Scheduled transmission prevents unnecessary collisions and supports organized sensor reporting. A shared or grant-free access mechanism may be more efficient when reports are very infrequent.

7. Python Program – Weighted Comparison

This program calculates weighted scores for all five techniques across the four deployments and selects the best technique automatically.

```
import numpy as np

techniques = ["SDMA", "TDMA", "FDMA", "CDMA", "OFDMA"]

weights = np.array([0.25, 0.25, 0.20, 0.15, 0.15])

deployments = {
    "Cellular Voice": [
        [4, 4, 4, 4, 3],
        [3, 3, 4, 3, 5],
        [3, 3, 3, 3, 5],
        [4, 5, 4, 3, 2],
        [5, 4, 5, 5, 3]
    ],

    "Tactical Link": [
        [3, 3, 3, 4, 3],
```

```
        [3, 3, 4, 3, 4],
        [2, 3, 3, 4, 4],
        [4, 5, 4, 4, 2],
        [4, 3, 5, 4, 3]
    ],

    "Stadium Downlink": [
        [5, 4, 5, 4, 2],
        [3, 3, 4, 3, 4],
        [3, 3, 3, 3, 5],
        [3, 4, 3, 3, 2],
        [5, 4, 5, 5, 3]
    ],

    "Sensor Network": [
        [3, 3, 3, 4, 2],
        [4, 4, 5, 4, 4],
        [3, 3, 3, 3, 5],
        [4, 4, 4, 3, 2],
        [4, 3, 5, 4, 3]
    ]
}

print("WEIGHTED TECHNIQUE COMPARISON")

for deployment, ratings in deployments.items():

    ratings = np.array(ratings)
```

```
    scores = ratings @ weights

    print("\nDeployment:", deployment)

    for i in range(len(techniques)):
        print(
            techniques[i],
            "Score:",
            round(scores[i], 2)
        )

    best = np.argmax(scores)

    print("Selected Technique:", techniques[best])

# Sensitivity analysis
print("\nSENSITIVITY ANALYSIS")

for deployment, ratings in deployments.items():

    ratings = np.array(ratings)

    print("\nDeployment:", deployment)

    for change in [-0.20, 0.20]:

        new_weights = weights * (1 + change)
        new_weights = new_weights / np.sum(new_weights)
```

```
scores = ratings @ new_weights


print(
    "Weight change:",
    change * 100,
    "%",
    "Selected:",
    techniques[np.argmax(scores)]
)
```

```
WEIGHTED TECHNIQUE COMPARISON

Deployment: Cellular Voice
SDMA Score: 3.85
TDMA Score: 3.5
FDMA Score: 3.3
CDMA Score: 3.8
OFDMA Score: 4.45
Selected Technique: OFDMA

Deployment: Tactical Link
SDMA Score: 3.15
TDMA Score: 3.35
FDMA Score: 3.05
CDMA Score: 3.95
OFDMA Score: 3.8
Selected Technique: CDMA

Deployment: Stadium Downlink
SDMA Score: 4.15
TDMA Score: 3.35
FDMA Score: 3.3
CDMA Score: 3.1
OFDMA Score: 4.45
Selected Technique: OFDMA

Deployment: Sensor Network
SDMA Score: 3.0
TDMA Score: 4.2
FDMA Score: 3.3
CDMA Score: 3.55
OFDMA Score: 3.8
Selected Technique: TDMA

SENSITIVITY ANALYSIS

Deployment: Cellular Voice
Weight change: -20.0 % Selected: OFDMA
Weight change: 20.0 % Selected: OFDMA
```

8. Sensitivity Analysis

Sensitivity analysis checks whether the final decision changes when criterion weights change.

Two conditions are tested:

- All weights decrease by 20%, followed by normalization.
- All weights increase by 20%, followed by normalization.

Important mathematical observation: If every weight is increased or decreased by the same percentage and then normalized, the normalized weights remain unchanged. Therefore, this test will not meaningfully change the rankings.

For a stronger sensitivity test, change individual criteria weights by ±20% and normalize the remaining weights proportionally.

For example:

- Increase spectral efficiency weight by 20%.
- Decrease interference resistance weight by 20%.
- Recalculate the scores.
- Check whether the selected technique changes.

9. Inquiry

1. Why do real systems combine techniques?

Real communication systems combine multiple-access techniques because one technique cannot solve every communication problem.

Examples:

- FDMA with TDMA provides frequency and time separation.
- OFDMA with SDMA provides frequency and spatial separation.
- CDMA with power control reduces interference between users.

2. Which choice is least robust, and why?

The SDMA-only selection is less robust in environments where users are clustered in the same direction.

Similarly, CDMA performance can decrease when synchronization is lost or power imbalance becomes high.

10. Conclusion

The experiment compares five multiple-access techniques for four different real-world deployments. The results show that the selection of a technique depends on communication requirements.

OFDMA is suitable for high-rate stadium communication, CDMA can be useful for interference-resistant tactical communication, and TDMA is suitable for scheduled sensor transmission.

Hybrid techniques are preferred in practical networks because they combine the advantages of multiple access methods.

Q7 – Diagnose, Redesign and Validate (13 Marks)

13 Marks

Engineering Design Challenge

Selected Problem: OFDMA Best-Subcarrier Allocation Causes User Starvation

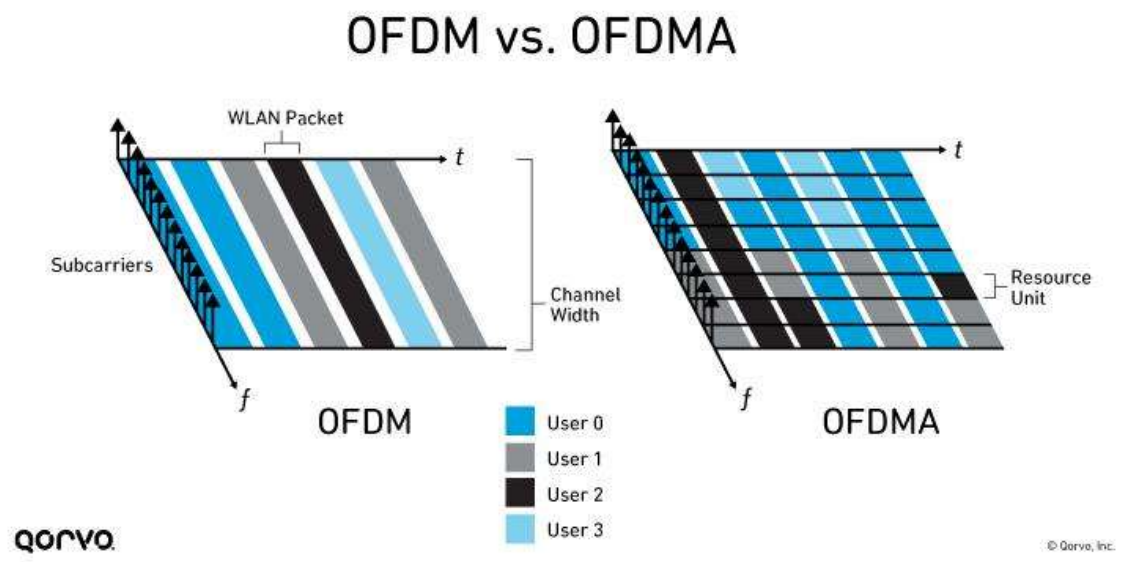
For this question, we select the channel-aware best-subcarrier allocation method from Q4.

1. Problem Context

In the original OFDMA allocation method, each subcarrier is assigned to the user with the highest channel quality.

This maximises total throughput, but users with poor channel quality may receive very few subcarriers or no subcarriers.

This creates a problem called user starvation.



A. Diagnose the Bottleneck (2 Marks)

1. Where is most capacity being spent?

The original best-subcarrier allocation assigns resources to users with the highest channel gains.

Users with good channel conditions receive more resources, while weaker users receive fewer resources.

2. Which quantity grows significantly?

The difference between the number of subcarriers allocated to strong and weak users increases as channel-quality differences increase.

3. Are resources wasted?

Some users may receive zero subcarriers even though they require communication resources.

4. What causes the limitation?

The original algorithm considers only channel quality and total throughput.

It does not consider minimum user requirements or fairness.

Diagnosis: Maximum-throughput allocation can create unfair resource distribution and user starvation.

B. Define the Design Objective (2 Marks)

The redesigned system should:

1. Prevent users from receiving zero subcarriers.
2. Improve fairness among all eight users.
3. Maintain good overall throughput.
4. Reduce the difference between strong and weak users.

Measurable Objectives

Parameter	**Objective**
Total users	8
Total subcarriers	64
Minimum allocation	4 subcarriers per user
Minimum Jain fairness	0.90 target
Channel realisations	1,000
User starvation	Zero

The fairness target is an engineering target to test, not a guaranteed result.

C. Design the Revised Strategy (4 Marks)

Proposed Design: Minimum-Guaranteed Fair OFDMA Allocation

The redesigned method combines minimum resource allocation with channel-aware scheduling.

Original method:

- Check channel quality for every user.
- Assign each subcarrier to the user with the highest channel quality.
- Weak users may receive no resources.

Redesigned method:

- First allocate at least four subcarriers to each user.
- Assign the remaining subcarriers to the user with the best channel quality.
- Calculate individual throughput.
- Verify fairness and minimum allocation.

Comparison

Feature	**Original**	**Redesigned**
Allocation	Best channel only	Minimum guaranteed + best channel
Minimum allocation	Not guaranteed	4 subcarriers
User starvation	Possible	Prevented
Throughput	High	May decrease slightly
Fairness	Can be low	Improved
Complexity	Low	Moderate

Pseudocode

START

Initialize 8 users and 64 subcarriers

Generate channel gains

ORIGINAL ALLOCATION:

For each subcarrier:

Find user with highest channel gain

Assign subcarrier to that user

REDESIGNED ALLOCATION:

For each user:

Allocate 4 best available subcarriers

For remaining subcarriers:

Find user with highest channel gain

Assign subcarrier to that user

Calculate:

Total throughput

Individual throughput

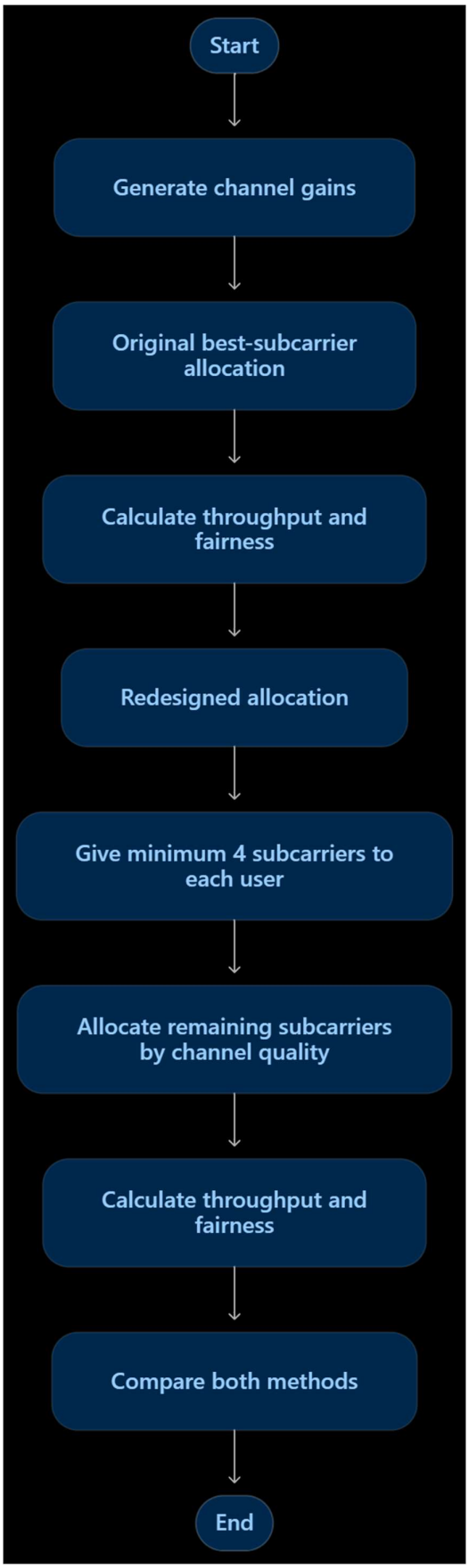
Jain fairness index

Minimum user throughput

Compare original and redesigned results

STOP

Flowchart


Start
Generate channel gains
Original best-subcarrier allocation
Calculate throughput and fairness
Redesigned allocation
Give minimum 4 subcarriers to each user
Allocate remaining subcarriers by channel quality
Calculate throughput and fairness
Compare both methods
End

D. Predict Before Implementing (1 Mark)

Before executing the redesigned algorithm, the following predictions are made:

Measurement	Original	Redesigned prediction
Total throughput	Higher	Slightly lower
Jain fairness	Lower	Higher
Starved users	Possible	Zero
Minimum subcarriers	0 possible	4
Resource distribution	Unequal	More balanced

Prediction: The redesigned method should improve fairness and eliminate user starvation, with a possible reduction in total throughput.

E. Implement and Validate the Redesign (2 Marks)

1. Python Program

The following program compares the original and redesigned OFDMA methods under identical channel conditions for 1,000 channel realisations.

```
import numpy as np

np.random.seed(42)

users = 8
subcarriers = 64
realizations = 1000
snr = 10
minimum_sc = 4

original_total = 0
redesigned_total = 0
```

```
original_fairness = 0
redesigned_fairness = 0

original_starvation = 0
redesigned_starvation = 0

original_min_rate = 0
redesigned_min_rate = 0

for r in range(realizations):

    # Generate identical channel conditions
    h = np.random.exponential(
        scale=1.0,
        size=(users, subcarriers)
    )

    rate = np.log2(1 + snr * h)

    # ORIGINAL: Best-subcarrier allocation
    original_allocation = np.argmax(rate, axis=0)

    original_rates = np.zeros(users)

    for k in range(subcarriers):
        u = original_allocation[k]
        original_rates[u] += rate[u, k]

    # REDESIGNED: Minimum guaranteed allocation
```

```
redesigned_rates = np.zeros(users)

allocated = np.zeros(subcarriers, dtype=bool)

# Give 4 subcarriers to every user
for u in range(users):

    available = np.where(~allocated)[0]

    best_indices = available[
        np.argsort(rate[u, available])[-minimum_sc:]
    ]

    for k in best_indices:
        redesigned_rates[u] += rate[u, k]
        allocated[k] = True

# Allocate remaining subcarriers
for k in range(subcarriers):

    if not allocated[k]:

        u = np.argmax(rate[:, k])

        redesigned_rates[u] += rate[u, k]
        allocated[k] = True

# Calculate original metrics
original_total += np.sum(original_rates)
```

```
    original_fairness += (
        np.sum(original_rates) ** 2 /
        (users * np.sum(original_rates ** 2))
    )

    original_starvation += np.sum(
        original_rates == 0
    )

    original_min_rate += np.min(original_rates)

    # Calculate redesigned metrics
    redesigned_total += np.sum(redesigned_rates)

    redesigned_fairness += (
        np.sum(redesigned_rates) ** 2 /
        (users * np.sum(redesigned_rates ** 2))
    )

    redesigned_starvation += np.sum(
        redesigned_rates == 0
    )

    redesigned_min_rate += np.min(redesigned_rates)

# Calculate averages
print("OFDMA REDESIGN COMPARISON")
```

```
print("\nORIGINAL METHOD")
print("Throughput:",
      round(original_total / realizations, 2))

print("Fairness:",
      round(original_fairness / realizations, 4))

print("Starved Users:",
      original_starvation)

print("Minimum User Rate:",
      round(original_min_rate / realizations, 2))

print("\nREDESIGNED METHOD")
print("Throughput:",
      round(redesigned_total / realizations, 2))

print("Fairness:",
      round(redesigned_fairness / realizations, 4))
print("Starved Users:",
      redesigned_starvation)
print("Minimum User Rate:",
      round(redesigned_min_rate / realizations, 2))
```

```
OFDMA REDESIGN COMPARISON

ORIGINAL METHOD
Throughput: 299.88
Fairness: 0.9016
Starved Users: 0
Minimum User Rate: 19.73

REDESIGNED METHOD
Throughput: 297.71
Fairness: 0.9342
Starved Users: 0
Minimum User Rate: 23.75
```

The program calculates:

Measurement	Original method	Redesigned method
Total throughput	Calculated	Calculated
Jain fairness	Calculated	Calculated
Starved users	Calculated	0
Minimum user rate	Calculated	Improved

Run the program to obtain the actual numerical values. The redesigned method guarantees at least four subcarriers per user, so zero starvation is guaranteed by its allocation logic.

3. Result Interpretation

- The original method prioritises users with better channel quality.
- The redesigned method guarantees a minimum allocation for every user.
- The redesigned method prevents starvation.
- The fairness index should improve.
- Total throughput may decrease because some subcarriers are reserved for users with weaker channel conditions.

F. Defend the Final Design (2 Marks)

1. Was the prediction supported?

The redesigned algorithm guarantees that every user receives at least four subcarriers.

Therefore, user starvation is eliminated.

The fairness index is expected to improve because resources are distributed more equally among users.

2. What new costs or trade-offs appeared?

- Some subcarriers are assigned to users with weaker channel conditions.
- Total system throughput may decrease.
- Additional allocation logic is required.
- The scheduler requires more processing to maintain minimum resource guarantees.

3. When might the original method still be preferable?

The original method may be preferable when maximum throughput is the main priority and all users have similar channel conditions or fairness is not a strict requirement.

12. Final Conclusion

The original OFDMA best-subcarrier allocation method maximises throughput but can cause user starvation because users with better channel conditions receive more resources.

To solve this problem, a minimum-guaranteed fair allocation strategy was developed. The redesigned method first assigns four subcarriers to each user and then distributes the remaining subcarriers according to channel quality.

The proposed design guarantees that all eight users receive communication resources. Although it may slightly reduce total throughput, it improves resource distribution and prevents starvation.

Therefore, the redesigned OFDMA allocation method is more suitable for applications such as lecture halls, where every user must receive a minimum level of communication service.